Grinta for Life — Member Agent
Technical Specification — v1.1

Six-layer information architecture for the member-facing AI

Author: Jay Crain
Date: May 2026
Version: 1.1

How to read this Spec

This document is the working technical specification for the Member Agent — the public-facing AI in the Grinta for Life program. It describes the six layers of information the agent operates on, the four operating moments where the agent interacts with members, the voice and governance enforcement that constrains the agent, the failure modes the architecture mitigates, and the build sequence to platform launch in mid-October 2026.

The audience is multi-purpose. A developer being hired reads this as the hand-off. An agency bidding on the build reads it as the basis for a statement of work. The internal team reads it as the architectural reference. The same content serves all three; each reader extracts what they need.

The Spec is built for iteration. The product experience team is iterating Tom Miller Member Journey v4.2; when Tom updates, sections of this Spec update with him. Section 9 names every Tom Miller dependency explicitly. Versioning convention: Tom v4.x maps to Spec v1.x; Tom v5.x maps to Spec v2.x.

Diagram — Member Agent Data Model (front-page reference):

The diagram color-codes the six layers — blue static (loaded once at agent init), orange dynamic (per-conversation), mustard Greg's contributions (Layer 2 sub-corpora), olive output (per-session). Solid arrows are required loads; dashed arrows are RAG retrieval and persistence. Each layer expands into a technical chapter in Section 3.

1. Executive Architecture Summary

1.1 The Member Agent's role

The Member Agent is the public-facing AI in Grinta for Life. It is the connective tissue between the framework (the 4Rs, the Atlas of 28 assets, the IDQ instrument, the ID Score), the member's longitudinal record (every IDQ retake, every Pulse Check, every gating asset completion), and the human community. Its job is to make every member interaction feel continuous, personal, and grounded in that member's specific story.

The Member Agent does not coach, prescribe, or substitute for human relationship. It listens, reflects, asks one question at a time, and routes members to the human community, the Direct tier coaches, the 988 Suicide and Crisis Lifeline, and the Founder Agent's drafted correspondence at the right moment. The Member Agent's most consequential function is knowing when to hand a member to a human — and doing it at the right moment.

This Spec defines how the agent does that work technically.

1.2 Reference architecture

The Member Agent runs on the platform reference architecture defined in Appendix C v1.3:

- Web app framework + hosting: Next.js on Vercel
- Data layer: Supabase (bundled Postgres, Auth, Storage, Realtime)
- Authentication: Supabase Auth (bundled with data layer)
- AI provider: Anthropic API (Claude family); provider abstraction layer in place for hedge
- CRM spine: HubSpot (relationship view; every contact, every campaign)
- Program spine: Supabase warehouse (every member's IDQ trajectory, gating asset completions, behavioral signals)

Developers proposing deviation from this reference architecture must justify the time and cost trade-off in writing inside the statement of work. Jay approves all deviations. The deviation policy is in Appendix C v1.3 Section C.13.5.

1.3 The six-layer information model

The Member Agent works with six distinct layers of information at every conversation. Three are static (loaded once at agent initialization or by RAG retrieval against a stable corpus); three are dynamic (loaded fresh per conversation per member); one is output (generated each session).

#	Layer	Summary
1	Agent Identity & Configuration	Voice rules, AI Governance prohibitions, Independence Guarantee posture, 988 protocol, reflect-and-route patterns, AI disclosure first line. The agent's "self." Loaded at agent init; constant across all conversations.
2	Program Knowledge Corpus	The 4Rs, the Atlas (28 assets), the 8 Doors, IDQ instrument, ID Score bands, SDT, Lorig & Holman, CDSMP, Self Checks from the book, plus Greg Welk's three named sub-corpora (Concept Map, 12 Science Checks, 7 Topical Syntheses). Retrieved by RAG based on member context.

3	Member Profile	Member ID, demographics, named Door, identity paragraph, structured intake data (athletic past, gap, right now, Reclaim List seed), tier, consent. Loaded per conversation for the specific member.
4	Member History	IDQ intake + every 60-day retake, Pulse Checks, gating asset completions and key outputs, behavioral signals, drift events, phase progression, cycle indicator. Loaded per conversation; summarized for context window efficiency with recent events included verbatim.
5	Conversation Context	Operating moment (Onboarding / IDQ Retake / Pulse Check / Ongoing Check-in), time since last interaction, recent dashboard state, pending hand-offs from Founder Agent or human team, crisis flags from prior conversations. Set per session.
6	Conversation Output	Transcript, profile updates, history entries, hand-offs triggered, flags for human review. Persisted back to the warehouse and HubSpot at end of session.

1.4 The four operating moments

The Member Agent's interactions with members fall into exactly four operating moments. Each moment has a defined system prompt slice, retrieval strategy, output expectation, and persistence pattern. Section 4 specifies each in detail.

- Onboarding — the structured intake conversation that produces the member identity paragraph and structured data object. Once per member per cycle.
- IDQ Retake — administering the 24-item Identity Distance Questionnaire every 60 days. Produces the ID Score, dimension scores, and the delta from baseline.
- Pulse Check — a 3–5 question survey between IDQ retakes, rotating IDQ dimension coverage. Lower-friction signal capture.
- Ongoing Check-in — open conversations any time, drawing on the member's full history. The agent's default mode when none of the above three are scheduled.

1.5 Versioning convention

This Spec moves in step with Tom Miller Member Journey. The product experience team iterates Tom Miller (currently v4.2); when Tom updates, the Spec sections that depend on Tom update with him. Section 9 maintains the dependency list. Convention:

- Tom Miller v4.x → Technical Spec v1.x (point releases for content refinement)
- Tom Miller v5.x → Technical Spec v2.x (major change, signals architectural revisit)

Anyone reading either document knows the compatibility from the version numbers alone. This is a deliberate operational choice to prevent the Spec and the canonical member experience from drifting out of sync.

2. Reference Architecture

2.1 Subdomain map

The platform comprises a public-facing marketing site, two custom application surfaces, and four functional-tool surfaces hosted at subdomains of grintaforlife.com. The map below is locked at October platform launch (mid-October 2026); pre-launch uses a subset of these surfaces with manual operations.

Subdomain	Purpose
grintaforlife.com	Public-facing marketing site (Webflow). Member application embedded as Typeform.
dashboard.grintaforlife.com	platform launch custom application — member home surface with hero centerpiece, ID Score, four IDQ dimensions chart, trajectory line, named Door, next-step CTA, identity history strip.
chat.grintaforlife.com	platform launch custom application — Member Agent conversation surface for onboarding, IDQ retakes, Pulse Check, ongoing check-ins, Cycle 2 renewal.
tovuti.grintaforlife.com	Tovuti LMS — gating asset views and Atlas asset delivery. Used in both pre-launch and platform launch.
community.grintaforlife.com	Circle community spaces — The Spark, The Ride, The Lounge, the Atlas spaces and their threads.
idq.grintaforlife.com	Typeform-hosted assessments during pre-launch (IDQ intake, Pulse Check, False Start Protocol form). Retires when platform launch absorbs these flows.

2.2 The two data spines

Two architectural spines hold the program together. HubSpot is the relationship spine (every contact, every campaign, every email). The Supabase warehouse is the program spine (every member's IDQ trajectory, gating asset completions, behavioral signals). They join via the HubSpot contact ID as foreign key.

HubSpot — relationship spine

Single source of truth for every person who has touched G4L. Visitor who signed up for the newsletter. Reader who took the public self-assessment. Member. Patron. Foundation contact. Prospective coach. Podcast guest. Press contact. Lifecycle stage tracking. Pipelines for Patron, foundation grants, and coach prospects. Newsletter publishing, lifecycle email, and transactional email in one platform for the Year 1 architecture.

Storage: HubSpot's cloud infrastructure. Standard data export and webhook integrations. No local replication.

Relationship to the Member Agent: HubSpot provides the contact ID and lifecycle stage to the agent at conversation start. The agent does not write directly to HubSpot; conversation outputs that update lifecycle stage flow through the Founder Agent's review queue and the warehouse.

Supabase warehouse — program spine

The data warehouse holds every Member's program-side record. IDQ intake and quarterly retakes. Pulse Check responses. gating asset completions and reflections. Behavioral signals (Window completions, miles logged, False Starts and re-entries). Drift events. Dashboard data flow. The anonymized research feed for IRB studies.

Storage: Postgres on Supabase, with row-level security enforcing AI Governance Framework data commitments at the database layer rather than at application level.

Relationship to the Member Agent: The warehouse is where the agent reads Layer 4 (Member History) from at every conversation and writes Layer 6 (Conversation Output) back to at every session end. The agent's persistence operations are warehouse-first, HubSpot-summary-second.

2.3 Provider Selection Rationale — Anthropic

The Member Agent runs on the Anthropic API. The selection rationale is in Appendix C v1.3 Section C.2; the working summary:

- Voice fidelity under tight system prompts. Anthropic's models hold disciplined voice across long conversations with less drift than alternatives at comparable price points.
- Refusal behavior aligned with Brand Standards prohibitions. The model refuses requests that conflict with the AI Governance Framework (diagnosing, raw score reveal, mental health response without 988 routing) without being over-restrictive in adjacent territory.
- Constitutional AI training and safety tooling. Aligned with G4L's emotional-safety commitments by design rather than by guardrails layered on top.
- Long-context handling. The agent operates with substantial Layer 2 retrieval plus full Layer 3 and summarized Layer 4 context. Anthropic's context windows handle this without aggressive truncation.

Trade-offs accepted: Pricing higher than OpenAI at comparable tiers. Smaller integration ecosystem. Newer enterprise relationships. Named and tracked.

Hedge: Provider abstraction layer in Phase 1 of the Platform Development Plan v4. The choice is deliberate, not permanent. A vendor evaluation against alternatives runs 90–180 days after October platform launch.

2.4 Cost ceiling and reliability commitments

The agent runs against six Consistency and Reliability Commitments from Platform Development Plan v4. Each is a mitigation against a known API-dependency risk. The Spec implements each in the layers and operating moments named below.

Commitment	Implementation in this Spec
Provider abstraction layer	Section 5.4 — agent runtime wraps Anthropic API calls in a provider-agnostic interface. Swap-in of an alternative provider is a configuration change, not a rewrite.
Graceful degradation	Section 6.2 — when the API is unavailable, the agent falls back to a templated experience that respects the Independence Guarantee (Section 6.3) and preserves member data continuity.
Voice drift monitor	Section 5.5 — 90-day cadence comparing agent outputs against Brand Standards exemplars. Drift triggers system prompt recalibration.

Account redundancy	Section 6.4 — two API account credentials, primary and failover. Rotation policy named.
Cost ceiling	Section 6.5 — per-conversation token budget and monthly aggregate cap. Alarms at 75% and 90% of monthly cap. Hard cap halts non-essential operations (social copy generation, batch operations) before member-facing operations.
Paper protocol	Section 6.3 — Independence Guarantee printable workflow for every gating asset. A member can complete the framework on paper if technology fails them. The Member Agent never gatekeeps the framework.

3. Layer-by-Layer Technical Specification

Each layer in the diagram expands into a technical chapter below. Sections follow a consistent shape: purpose, data structures, storage location, loading and retrieval pattern, update and persistence rules, voice and governance enforcement at this layer, failure modes, and dependencies.

3.1 Layer 1 — Agent Identity & Configuration

Purpose

Layer 1 is the agent's "self." It defines who the agent is, how it speaks, what it refuses to do, and what it routes elsewhere. Layer 1 is loaded once at agent initialization and remains constant across all conversations. It does not vary by member.

Contents

- **Voice rules** — three Jay registers from Brand Standards v1.2, with the Member-facing register active for the Member Agent (warm, relational, member-paced, listens before reflecting, one question at a time, no "journey," no "I hear you," no "amazing," short sentences, locked vocabulary capitalized on first mention)
- **AI Governance Framework prohibitions** — never diagnose, never reveal raw ID Score without band label and human context, never address mental health disclosure directly (route to 988), never suggest programs/tiers/upgrades, never operate outside the G4L voice and 4R framework, never substitute for human coaching at the Direct tier, never store or reference info beyond explicit consent, never continue a conversation after a stop signal
- **988 protocol** — detection patterns for distress signals (hopelessness, self-harm references, acute grief, crisis language); response template (warm acknowledgment, no counsel, immediate 988 reference with phone number, no follow-up questions on the disclosure); escalation flag to human team within 24 hours

- **Reflect-and-route patterns** — explicit routing tables for science questions (route to Greg's Concept Map + Science Check + next Greg AMA), commercial questions (route to founder, not auto-answer), coaching questions (route to Direct tier if applicable), and community questions (route to Circle space)
- **AI disclosure first line** — the locked text shown as the first message of every member's first interaction with the Member Agent: *"This conversation is guided by AI. Everything you share shapes your G4L experience and is handled with the same care you'd expect from a person. You can stop at any time."*
- **Independence Guarantee posture** — the agent is a service, not a requirement. Members who prefer a text-based profile form get the form without penalty. Members who skip an IDQ retake are not gatekept. Paper protocol available for every gating asset.

Storage

Layer 1 is implemented as the agent's system prompt. The system prompt is treated as production code, version-controlled in the application repository, reviewed quarterly under the AI Governance Framework. The prompt is not stored in the warehouse; it is loaded from the application code at agent initialization.

Implementation pattern (Next.js Server Component / Server Action wrapping the Anthropic API call):

```
// /lib/agent/system-prompt.ts
export const MEMBER_AGENT_SYSTEM_PROMPT = `
You are the Grinta for Life Member Agent.
VOICE
- Brand Standards v1.2 Member-facing register.
- Warm, relational, member-paced. Listen before reflecting.
- Short sentences. One question at a time.
- Never the word "journey." Never "I hear you." Never "amazing."
- Locked vocabulary capitalized on first mention.
GOVERNANCE PROHIBITIONS (non-negotiable)
- Never diagnose, label, or pathologize a member's experience.
- Never reveal a ID Score as a raw number without band label
  and plain-language interpretation.
- Never address mental health disclosure directly. Route to 988
  immediately with: "If you're in crisis, please call or text 988
  for the Suicide and Crisis Lifeline."
- Never suggest programs, tiers, upgrades, or commercial offerings.
- Never substitute for human coaching at the Direct tier.
REFLECT-AND-ROUTE
- Science questions: reflect Greg's content in G4L voice; route to
  Concept Map / Science Check / next Greg AMA. Do NOT
  impersonate Greg.
- Commercial questions: route to founder; do not auto-answer.
- Coaching questions: route to Direct tier if member is on tier.
- Community questions: route to relevant Circle space.
INDEPENDENCE GUARANTEE
- The agent is a service, not a requirement. Members can skip
  any interaction. Paper protocol available for every gating asset.
`;
```

Loading

The system prompt is prepended to every API request as the system message. Layers 2 through 5 are appended as user-side context messages or via tool calls. The system prompt itself does not change within a session.

Voice and governance enforcement at this layer

Layer 1 is the primary enforcement surface for both voice fidelity and governance prohibitions. The system prompt declares the rules; the model's training and the API's refusal behavior carry them. Section 5 covers the enforcement architecture in detail.

Failure modes

System prompt drift across edits. When the system prompt is edited (new prohibition added, new register specification, etc.), regression tests against locked voice exemplars verify the agent still passes voice fidelity checks before the prompt update ships.

Refusal over-reach. The model may refuse legitimate member questions if the system prompt's prohibition language is too broad. The 90-day voice review cadence reviews refusals as well as outputs.

Locale-specific 988 limitations. 988 is US-only. The Spec assumes US members at platform launch; international expansion adds a locale-specific routing table to Layer 1.

Dependencies

Layer 1 depends on Brand Standards v1.2 (and Brand Standards v1.3 when finalized) for voice rules. Depends on AI Governance Framework v1.2 for prohibitions and emotional safety protocol. Independent of Tom Miller Member Journey v4.2 — Tom v4.x iterations do not change Layer 1 unless Brand Standards changes with them.

3.2 Layer 2 — Program Knowledge Corpus

Purpose

Layer 2 holds everything the agent knows about Grinta for Life itself. The framework, the Atlas, the IDQ, the science underneath each design choice. Layer 2 is shared across all members; it is static in the sense that it does not vary by member, dynamic in the sense that the corpus updates when the framework updates.

Contents

Layer 2 is structured into two tiers — the core program corpus and the three Greg sub-corpora.

Core program corpus

- The 4Rs framework definitions — Reconnect, Rewire, Rebuild, Reclaim. Each phase with its Layers (Distance/Excavation/Spark; Awareness/Visualization/Hardiness; Foundation/Structure/Elevation; Emergence/Extension/Legacy) and assets.
- The 8 Doors — The Career, The Marriage, The Empty Nest, The Cliff, The Body, The Diagnosis, The Loss, The Aging Parents. Each with a descriptor and the framework's reflect-and-route guidance.
- The Atlas — 28 assets — 12 gating assets + 16 Depth-tier asset (retired tier name; now just "asset")s. Each with module number, phase code, layer, purpose, tier, and Independence Guarantee paper protocol pointer.
- The IDQ instrument — 24 items across 4 IDQ dimensions (Physical, Identity, Relational, Forward), 6 items per dimension. Each item with stem, Likert anchors, and dimension assignment. Scoring formula: ID Score = (Total Raw Score / 120) × 100.
- The ID Score bands — Starting Line (below 40), The Fade (40–59), Drifting (60–79), In Flow (80–100). Each band with plain-language interpretation.
- Self-Determination Theory mapping — autonomy, competence, relatedness mapped onto the 4Rs at the structural level.
- Lorig & Holman self-management model — three tasks (medical, role, emotional), six skills (problem-solving, decision-making, resource utilization, patient-provider partnership, action planning, self-tailoring), CDSMP operational adaptation.
- Identity Synthesis pattern (Option A) — synthesize-propose-confirm. The agent synthesizes from the member's own words, proposes a noun, asks the member to confirm, commits only on confirmation.
- Cycle 2 three-path Identity Check-In — keep the prior noun, hold a compound, adopt a new noun. The framework supports all three.
- The Seven Minutes — the headline brand phrase; metaphorical place where getting started is what matters.
- Self Checks from the book — the in-book self-assessments. Cross-referenced into the Atlas and surfaced by the agent when relevant. (Integration is pending Brand Standards v1.3 incorporation.)

Greg sub-corpora — three named, retrieved by RAG

- Concept Map — G4L-keyed. 10–15 pages organized by Phase / Layer / gating asset. Greg-authored. The reference document the agent uses for the "why does the framework work this way" reflect-and-route conversations. Sub-corpus identifier: `greg.concept_map`.
- 12 Science Checks. ~200 words each, one per gating asset, Greg-signed. Retrieved when the agent encounters the corresponding gating asset in a member's conversation. Sub-corpus identifier: `greg.science_check_zones[flagship_code]`.
- 7 Topical Syntheses. 1,500–3,000 words each on specific topics (behavior change, hardiness, shortcut trap, SDT in midlife, PA assessment, Fuel Plan, the shortcut trap). Retrieved when conversation goes deep on a subject. Sub-corpus identifier: `greg.topical_syntheses[topic]`.

The full structure of the Greg sub-corpora is documented in the G4L × Greg Welk Authoring Brief v1.0 (companion document).

Storage

Layer 2 lives in the Supabase warehouse as two related tables plus a vector index:

```
— program_knowledge — structured reference content
create table program_knowledge (
 id           uuid primary key default gen_random_uuid(),
 doc_type      text not null,      — 'asset' | 'door' | 'dimension' | 'concept_map' | 'science_check' |
'topical_synthesis' | 'self_check' | 'reference'
 identifier     text not null,      — e.g. 'R-1', 'door.the_career', 'greg.science_check.R-1'
 title         text not null,
 body          text not null,      — markdown content
 phase         text,              — 'Reconnect' | 'Rewire' | 'Rebuild' | 'Reclaim' | null
 layer         text,              — e.g. 'Distance' | 'Excavation' | 'Spark'
 tier          text,              — 'asset' | 'depth'
 version       text not null,      — e.g. '1.0'
 effective_at   timestamptz not null default now(),
 retired_at     timestamptz,
 created_by     text not null,
 metadata      jsonb default '{}'::jsonb,
 unique (identifier, version)
);
— program_knowledge_embedding — for RAG retrieval
create table program_knowledge_embedding (
 knowledge_id   uuid references program_knowledge(id) on delete cascade,
 embedding      vector(1536) not null,  — pgvector extension
 model_version   text not null,
 created_at     timestamptz not null default now()
);
create index on program_knowledge_embedding using ivfflat (embedding
vector_cosine_ops);
```

Loading and retrieval

Layer 2 is large. Loading the entire corpus into every conversation context would exhaust the API context window. Retrieval is selective:

- *Always-loaded core.* The 4Rs framework summary, the 8 Doors descriptors, the IDQ instrument structure, the ID Score bands, the Identity Synthesis pattern, and the AI disclosure are loaded into every conversation as part of the system prompt's program-knowledge slice. Compact, summarized; full text retrieved when needed.
- *RAG-retrieved by member context*. When a member's conversation enters a specific gating asset (signaled by Layer 4 history or Layer 5 context), the agent retrieves: (a) the gating asset's program_knowledge entry, (b) the corresponding Greg Science Check (if exists), (c) any Topical Synthesis tagged for that gating asset's domain. Retrieval is semantic similarity against an embedding of the member's recent conversation turn plus a context summary.
- *RAG-retrieved by topic.* When a member's conversation surfaces a topic (asking about science, about a Door, about a self-check from the book), the agent retrieves the relevant Concept Map section, Topical Synthesis, or Self Check via semantic search.
- *Top-K retrieval.* Default K = 3 per retrieval call. Trade-off between context window cost and corpus coverage. Tunable; see Section 8 open decisions.

Update and persistence rules

Layer 2 is read-only from the agent's perspective. The agent never writes to Layer 2 in a session. Updates happen out-of-band, via:

- Brand Standards v1.3 finalization → updates to voice rules, locked vocabulary, deprecated language; cascades into Layer 1 system prompt and Layer 2 program-knowledge entries
- Greg's Concept Map, Science Checks, and Topical Syntheses authoring → new program_knowledge rows with version increments; embedding regeneration; deployment via migration
- Self Checks integration → new program_knowledge rows tagged `doc_type = 'self_check'`; cross-referenced to Atlas entries via metadata
- Framework refinement → updates to Atlas asset entries; version-incremented rows; the retired version remains queryable for audit

Voice and governance enforcement at this layer

Layer 2 enforces the voice boundary by content: Greg-authored entries (Concept Map, Science Checks, Topical Syntheses) are tagged as Greg's voice; the system prompt instructs the agent to reflect these in G4L voice and route to them, never to impersonate Greg. The agent treats Greg-authored content as source material to refer to, not as a script to perform.

Citation integrity is enforced by metadata: every Greg-authored entry includes citation pointers in the `metadata` JSON. When the agent surfaces information drawn from Greg's content, it can also surface the citation pointer if asked.

Failure modes

RAG retrieval misses the right context. Mitigation: instrument retrieval calls; review weekly during pre-launch; tune K and embedding model based on miss patterns.

Embedding model version drift. When the embedding model updates, the entire corpus needs re-embedding. The `program_knowledge_embedding.model_version` field tracks this. Migration policy: dual-run old and new embeddings during transition; cutover when new model passes regression tests.

Corpus staleness. If a framework element updates and the corpus is not refreshed, the agent surfaces outdated content. Mitigation: every program_knowledge update triggers a deployment notification; weekly review during pre-launch; automated alerting if any entry's `effective_at` is older than the latest framework version.

Dependencies

Layer 2 depends on Brand Standards v1.2/v1.3 (voice rules carried into program_knowledge content). Depends on the Atlas (Appendix E v1.2) for the 28 asset entries. Depends on the IDQ instrument design and Identity Synthesis pattern (locked). Depends on Greg's authoring deliverables per the Authoring Brief v1.0. Tom Miller dependency: the Self Checks integration, Cycle 2 three-path Identity Check-In, and the Atlas gating asset sequencing are reflected in Tom v4.2; Tom v4.x updates may add Self Checks placement details that cascade into Layer 2 program_knowledge entries.

3.3 Layer 3 — Member Profile

Purpose

Layer 3 holds the structured, semi-static profile of a specific Member. It loads at the start of every conversation with that member and provides the agent with the member's identity, named Door, intake cont xt, tier, and consent posture. Layer 3 changes infrequently — updates happen when the member edits their identity noun, advances cycle, changes tier, or updates consent.

Contents and schema

```
— member_profile — one row per Member
create table member_profile (
  member_id           uuid primary key default gen_random_uuid(),
  hubspot_contact_id    text not null unique,        — foreign key to HubSpot CRM
  display_name         text not null,
  email               text not null,
  age                 int,
  location_city        text,
  location_state       text,
  named_door          text,                    — one of the 8 Doors
  identity_noun        text,                    — e.g. 'ATHLETE', 'FATHER'
  identity_paragraph    text,                    — Member Agent onboarding output
  intake_athletic_past   text,                    — structured intake chapter 1
  intake_gap           text,                    — structured intake chapter 2
  intake_right_now      text,                    — structured intake chapter 3
  reclaim_list_seed     jsonb,                   — 7-item list from Identity Synthesis
  membership_tier       text not null,             — 'foundation' | 'work' | 'direct'
  sms_opt_in           boolean default false,
  ai_consent_granted_at  timestamptz,
  cycle_indicator       int default 1,             — 1, 2, 3...
  active              boolean default true,
  created_at           timestamptz not null default now(),
  updated_at           timestamptz not null default now()
);
create index on member_profile (hubspot_contact_id);
create index on member_profile (membership_tier) where active = true;
```

Loading

At conversation start, the agent runtime loads Layer 3 with a single keyed query:

```
select * from member_profile
where hubspot_contact_id = $1 and active = true;
```

The full profile is included in the conversation context. It is compact (typically under 2KB) and used in every operating moment.

Update and persistence rules

Layer 3 updates are explicit and bounded. The agent writes updates only via the structured output channel; ad-hoc field mutations are not permitted.

- *Identity noun update* — only via the Cycle 2 three-path Identity Check-In (keep, hold compound, adopt new). The agent proposes; the member confirms; the update lands.
- *Reclaim List seed update* — additions allowed during the Reconnect phase; locked at Identity Excavation completion; revisable at Cycle 2 onset.
- *Identity paragraph refinement* — the agent may propose refinements during ongoing check-ins; the member confirms before commit.
- *Named Door change* — rare; only when a new Door opens (Cycle 2 onset). The Member Agent does not change Door unilaterally.
- *Tier change* — never written by the Member Agent. Tier changes flow through the Founder Agent's review queue and HubSpot.
- *Consent update* — written immediately when the member revokes or grants. The agent honors revocation in the same conversation it is given.

All updates emit a row in `member_profile_audit` (append-only) and a corresponding HubSpot lifecycle event.

Voice and governance enforcement at this layer

Layer 3 is where the synthesize-propose-confirm pattern lives. The agent never commits a member-facing identity label without explicit member confirmation. Section 5.2 details the implementation.

Consent is enforced at the database layer via Supabase row-level security: the agent cannot read fields the member has not consented to share. If `ai_consent_granted_at` is null, the agent operates in a degraded mode that respects the Independence Guarantee (text-based profile form, no conversational intake).

Failure modes

Profile load failure. If the warehouse is unreachable, the agent falls back to a minimal-context conversation that asks for the member's name and named Door inline. The member is not blocked; the conversation continues with reduced personalization. Section 6.2 covers the degradation pattern.

HubSpot contact ID drift. If the HubSpot ID changes (rare, but possible during data migrations), Layer 3 includes a `previous_hubspot_contact_id` migration table for reconciliation.

Dependencies

Layer 3 depends on the 8 Doors definition (Layer 2 / Brand Standards). Depends on the Identity Synthesis pattern (Layer 2). Tom Miller dependency: the structured intake chapters (athletic past, gap, right now) reflect Tom v4.2 Section D.1. If Tom v4.x adds or restructures the intake chapters, the `intake_*` schema fields and the corresponding system prompt slice update with him.

3.4 Layer 4 — Member History

Purpose

Layer 4 is the longitudinal record of a member's work inside the framework. It grows over time. Every IDQ retake, every Pulse Check, every gating asset completion with its key outputs, every behavioral signal, every drift event, every phase transition. Layer 4 is what makes the Member Agent feel continuous rather than transactional.

Layer 4 is the largest dynamic layer and the most operationally sensitive. The schema is shaped to support both the agent's per-conversation needs and the IRB-overseen research feed.

Contents and schema

Layer 4 is a set of related tables, one per entity type. Each row carries the `member_id` foreign key plus a timestamp and a `cycle_indicator` so cohort-level and longitudinal queries are clean.

```
— IDQ retakes (baseline + every 60-day retake)
create table idq_retake (
 retake_id      uuid primary key default gen_random_uuid(),
 member_id      uuid not null references member_profile(member_id),
 cycle_indicator int not null,
 sequence_no    int not null,            — 0 = baseline, 1, 2, 3...
 taken_at       timestamptz not null,
 responses      jsonb not null,          — 24 items, 1-5 Likert each
 physical_score  int not null,           — 0-25
 self_score  int not null,               — 0-25
 social_score int not null,              — 0-25
 outlook_score   int not null,           — 0-25
 id_score    int not null,               — 0-100 composite
 — bands retired per May 2026 cascade; movement shown via direction arrow + signed delta + raw
number
 delta_from_baseline int,                — null on baseline
 delta_from_previous int,                — null on baseline
 unique (member_id, cycle_indicator, sequence_no)
);
— Pulse Checks
create table pulse_check (
 pulse_id       uuid primary key default gen_random_uuid(),
 member_id      uuid not null references member_profile(member_id),
 cycle_indicator int not null,
 taken_at       timestamptz not null,
 dimension_focus text not null,          — which IDQ dimension rotated in
 responses      jsonb not null,          — 3-5 items
 signal_strength int                     — agent-computed; 0-100
);
— gating asset completions
```

```
create table flagship_completion (
  completion_id   uuid primary key default gen_random_uuid(),
  member_id       uuid not null references member_profile(member_id),
  cycle_indicator int not null,
  flagship_code   text not null,          — 'R-1' | 'R-4' | … | 'C-5'
  completed_at    timestamptz not null,
  outputs         jsonb not null,         — asset-specific structured output
  reflection      text,                   — member's reflection if captured
  unique (member_id, cycle_indicator, flagship_code)
);
— Behavioral signals (low-frequency event stream)
create table behavioral_signal (
  signal_id       uuid primary key default gen_random_uuid(),
  member_id       uuid not null references member_profile(member_id),
  cycle_indicator int not null,
  signal_type     text not null,          — 'check_in' | 'miles_logged' | 'false_start' | 're_entry' |
'window_completion' | …
  occurred_at     timestamptz not null,
  payload         jsonb,
  index (member_id, signal_type, occurred_at desc)
);
— Drift events
create table drift_event (
  event_id        uuid primary key default gen_random_uuid(),
  member_id       uuid not null references member_profile(member_id),
  cycle_indicator int not null,
  detected_at     timestamptz not null,
  trigger         text not null,          — 'two_check_ins_missed' | 'flagship_stalled_14d' | …
  re_engagement_sent boolean default false,
  re_engagement_sent_at timestamptz,
```

```
  recovered_at    timestamptz               — null until member re-engages
);
— Phase transitions
create table phase_transition (
  transition_id   uuid primary key default gen_random_uuid(),
  member_id       uuid not null references member_profile(member_id),
  cycle_indicator int not null,
  from_phase      text,                     — null on first entry
  to_phase        text not null,            — 'Reconnect' | 'Rewire' | 'Rebuild' | 'Reclaim'
  transition_at   timestamptz not null,
  gating_flagship text not null             — the gating asset completion that gated the transition
);
```

Loading and retrieval

Layer 4 grows. A member at Day 360 has six IDQ retakes, twelve Pulse Checks, twelve gating asset completions, dozens of behavioral signals, and a handful of drift events and phase transitions. Loading all of it verbatim into every conversation context is wasteful and rapidly exhausts the API context window.

The retrieval pattern is a summarized + recent-events hybrid:

- *Summarized rollup* — at conversation start, the agent runtime computes a compact summary: current phase, current cycle, latest ID Score and band, dimension trajectory (latest vs. baseline), most recent gating asset completed, behavioral streak status, any active drift event, any recent re-engagement note. ~500–800 tokens.
- *Recent events verbatim* — the last 30 days of behavioral signals, the last 2 Pulse Check responses, and the most recent gating asset reflection are loaded verbatim. ~500–1,500 tokens.
- *Deep retrieval on demand* — if the conversation surfaces a question about an earlier event ("how did I feel at my last retake?"), the agent can issue a Layer 4 query through a structured tool call. The result is loaded into the context for that turn only.

The summarized rollup is computed by a deterministic function in the agent runtime, not by the model. The model receives the rollup as fact, not as interpretation.

```
// /lib/agent/history-rollup.ts
export async function computeHistoryRollup(memberId: string): Promise<HistoryRollup> {
 const [latestIdq, idqTrajectory, latestgating asset, activeDrift, recentSignals]
  = await Promise.all([
   getLatestIdq(memberId),
   getIdqTrajectory(memberId),
   getLatestgating asset(memberId),
   getActiveDrift(memberId),
   getRecentSignals(memberId, days=30),
  ]);
 return {
  current_phase: derivePhase(latestgating asset),
  current_cycle: latestIdq?.cycle_indicator ?? 1,
  id_score: latestIdq?.id_score,
  band: latestIdq?.band,
  dimension_trajectory: idqTrajectory,   // baseline → latest per dimension
  most_recent_flagship: latestgating asset,
  active_drift: activeDrift,
  recent_signals_summary: summarize(recentSignals),
 };
}
```

Update and persistence rules

Layer 4 is append-mostly. Rows are created at the corresponding event; existing rows are not edited in normal operation. Two exceptions:

- Drift event recovery. When a drifting member re-engages, the `recovered_at` field updates on the drift event row. The event itself remains the historical record.
- IDQ retake correction. Rare — if a member contacts support to correct an IDQ response error within 24 hours, the row is corrected with a `corrected_at` and the original responses retained in a `corrections` jsonb field.

Every Layer 4 write also emits a HubSpot custom event for the member's contact record. This keeps the relationship spine (HubSpot) and the program spine (warehouse) in sync without requiring HubSpot to hold the full record.

Voice and governance enforcement at this layer

Layer 4 is the source of the data the agent reflects back to members. The AI Governance Framework prohibition on raw score reveal means the agent never returns a ID Score without the band label and a plain-language interpretation drawn from Layer 2 band entries. The history rollup function attaches the band label to every IDQ entry before the rollup is handed to the model.

Research consent is enforced at the warehouse boundary. The IRB-anonymized research feed reads from Layer 4 with member IDs replaced by hashed pseudonyms; the agent never queries the research feed.

Failure modes

Rollup function staleness. If the rollup is computed at conversation start and the conversation runs long, late-arriving events (a new IDQ submission mid-conversation) are missed. Mitigation: subscribe the agent to warehouse events for the active member; refresh the rollup if a relevant event arrives mid-conversation.

Cycle boundary ambiguity. Cycle transitions happen at C-7 Cycle Closing completion. If a member skips C-7 and starts new Reconnect work, the system needs to detect cycle onset. Mitigation: explicit Cycle 2 onboarding entry point in the conversation surface; agent prompts member to confirm cycle transition.

Behavioral signal flood. A member's wearable could in principle emit thousands of signals per day. Mitigation: signal ingestion has a rate limit and aggregation pre-step; the agent never sees raw signals, only the daily aggregate.

Dependencies

Layer 4 depends on the IDQ instrument structure (Layer 2). Depends on the 12 gating asset codes (Atlas / Layer 2). Depends on the cycle indicator convention (locked). Tom Miller dependency (heavy): the gating asset completion outputs schema is shaped to match Tom v4.2's documented outputs (Reclaim List items, anchor statements, False Start logs, Adventure Plan, Success Story). If Tom v4.x adds new gating asset outputs or restructures existing ones, the `outputs` jsonb schema needs migration. Section 9 carries the explicit dependency list.

3.5 Layer 5 — Conversation Context

Purpose

Layer 5 holds the per-session context that frames a specific conversation. It does not vary by member's underlying state (that's Layers 3 and 4); it varies by the moment in time the conversation is happening. Layer 5 answers: what is this conversation for, what just happened, what is pending, what should the agent be alert to.

Contents

- *Operating moment* — Onboarding / IDQ Retake / Pulse Check / Ongoing Check-in. Set when the conversation surface routes the member to the agent. Section 4 details each.
- *Time since last interaction* — seconds since the most recent agent conversation; signals whether this is a returning session or a long-absence return.
- *Time since last IDQ* — days since the last IDQ retake; signals whether an IDQ is due.
- *Recent dashboard state* — the member's dashboard reading as of the moment they opened the chat. Hero centerpiece, ID Score, latest gating asset surfaced.
- *Pending hand-offs from Founder Agent* — drafted welcome, milestone note, drift re-engagement, retake commentary waiting in the queue; the Member Agent does not send these but knows about them and can reference them appropriately.
- *Crisis flags from prior conversations* — if a prior session flagged distress, this conversation surfaces the flag for the agent to handle with the 988 protocol top of mind.
- *Entry context* — what surface the member came from (dashboard, email link, SMS link, direct URL); informs the agent's opening posture.

Storage

Layer 5 is a transient session object, not a persistent table. It is constructed at conversation start by the agent runtime and discarded at session end. The events that populate it draw from Layer 3 (profile), Layer 4 (recent events), and the conversation surface's entry context.

```
// /lib/agent/conversation-context.ts

export interface ConversationContext {

  operating_moment: 'onboarding' | 'idq_retake' | 'pulse_check' | 'ongoing_checkin';

  member_id: string;

  session_id: string;

  started_at: Date;

  seconds_since_last_session: number | null;

  days_since_last_idq: number | null;

  idq_retake_due: boolean;

  pulse_check_due: boolean;

  recent_dashboard_state: DashboardSnapshot;

  pending_founder_agent_drafts: FounderAgentDraft[ ];

  crisis_flags: CrisisFlag[ ];

  entry_context: 'dashboard' | 'email_link' | 'sms_link' | 'direct_url';

}
```

Loading

Layer 5 is computed at conversation start, before the first model call. Inputs:

- Operating moment from the conversation surface routing
- Member ID from Supabase Auth session
- Recent events from Layer 4 (`getRecentSignals`, `getLatestIdq`, `getActiveDrift`)
- Dashboard state snapshot from the dashboard application state
- Founder Agent queue from the warehouse `founder_agent_draft` table
- Crisis flags from the warehouse `conversation_flag` table

The session object is included in the agent's context window as a structured user-side message titled `SESSION_CONTEXT`. The model treats it as fact, not as interpretation.

Update and persistence rules

Layer 5 updates during the session as the conversation progresses. Examples:

- Member surfaces a crisis signal → `crisis_flags` updates immediately; 988 protocol activates
- Founder Agent sends a queued draft mid-conversation → `pending_founder_agent_drafts` updates; agent can acknowledge to member if relevant
- Member completes an action during the session (submits Pulse Check, requests an IDQ retake) → session state updates; persisted to Layer 4 at end of session

Session state is not persisted as a long-term record. The conversation transcript is (Layer 6); the in-flight session state is not.

Voice and governance enforcement at this layer

Layer 5 is where operating-moment-specific prompt slices attach. The system prompt's base (Layer 1) is constant; the operating-moment slice is appended based on `operating_moment`. Section 4 details each operating moment's prompt slice.

Crisis flag handling at Layer 5 is the highest-priority enforcement. When a flag is present, the agent's first turn references the prior disclosure with the 988 protocol's warm acknowledgment pattern. Section 5.3 details the protocol.

Failure modes

Stale dashboard state. If the member opens chat from a dashboard tab that's been open for hours, the dashboard snapshot is stale. Mitigation: dashboard state is refreshed on chat-surface load; if refresh fails, the agent's opening turn confirms the member's current understanding ("I'm seeing your ID Score at X — does that match what's on your dashboard?").

Operating moment mis-routing. If the conversation surface routes a member to IDQ Retake when the member is actually overdue for Pulse Check, the agent detects the mismatch and re-routes. The agent prefers what's actually due over what the surface routed.

Dependencies

Layer 5 depends on the four operating moments (locked). Depends on the conversation surface routing logic (chat.grintaforlife.com). Depends on Founder Agent queue state. Tom Miller dependency: the entry context patterns and the conversation surface's routing logic reflect Tom v4.2's documented session triggers. If Tom v4.x adds new session triggers, Layer 5's operating-moment enumeration and routing logic update.

3.6 Layer 6 — Conversation Output

Purpose

Layer 6 is everything the agent produces in a session: the conversation transcript, profile updates, history entries, hand-offs triggered, flags raised. Layer 6 is the boundary between the agent's runtime and the rest of the program. Outputs persist back to the warehouse and HubSpot; the session ends; the conversation surface clears its state.

Contents

- Conversation transcript — every member turn and every agent turn, with timestamps and turn indices. The transcript is the primary audit-trail asset for the AI Governance Framework.
- Profile updates — Layer 3 deltas requested during the session (identity paragraph refinement, Reclaim List additions, consent changes)
- History entries — Layer 4 rows created during the session (gating asset completion, IDQ retake submission, Pulse Check submission, behavioral signal, drift event recovery)
- Hand-offs triggered — events emitted to other systems (Founder Agent welcome draft request, coach notification for Direct tier moment, 988 routing for crisis, Circle community routing for peer connection)
- Flags raised — markers for human review (voice drift concern, prohibition near-violation, governance edge case)

Storage and schema

```
— conversation_session — one row per session
create table conversation_session (
 session_id       uuid primary key default gen_random_uuid(),
 member_id        uuid not null references member_profile(member_id),
 operating_moment  text not null,
 started_at       timestamptz not null,
 ended_at         timestamptz,
 entry_context     text,
 model_version     text not null,         — e.g. 'claude-sonnet-4-6'
 total_tokens      int,
 total_cost_cents  int,
 flag_count       int default 0
);
— conversation_turn — every turn in a session
create table conversation_turn (
 turn_id          uuid primary key default gen_random_uuid(),
 session_id       uuid not null references conversation_session(session_id) on delete cascade,
 turn_index       int not null,
 role            text not null,         — 'member' | 'agent' | 'system'
 content          text not null,
 occurred_at       timestamptz not null,
 tokens_used       int,
 unique (session_id, turn_index)
);
— conversation_handoff — events emitted to other systems
create table conversation_handoff (
 handoff_id       uuid primary key default gen_random_uuid(),
 session_id       uuid not null references conversation_session(session_id),
 handoff_type      text not null,         — 'founder_agent_draft' | 'coach_notify' | '988_route' |
'community_route'
```

```
 triggered_at     timestamptz not null,
 payload         jsonb,
 resolved_at      timestamptz,          — when downstream system acked
 resolved_status   text                — 'sent' | 'failed' | 'queued'
);
— conversation_flag — markers for human review
create table conversation_flag (
 flag_id         uuid primary key default gen_random_uuid(),
 session_id       uuid not null references conversation_session(session_id),
 flag_type        text not null,         — 'voice_drift' | 'near_prohibition' | 'governance_edge' |
'crisis_signal'
 raised_at        timestamptz not null,
 context         jsonb,
 reviewed_at      timestamptz,
 reviewed_by      text,
 resolution       text
);
```

Persistence pattern

Output persistence happens in three windows during the session, not only at session end. This protects against mid-session API failures losing data and enables the dashboard to update in near-real-time.

- Per-turn. Every conversation turn writes a `conversation_turn` row immediately after the model returns. The transcript is durable as soon as it exists.
- Per-action. When the member completes a structured action mid-conversation (submits IDQ, completes gating asset reflection, updates identity noun), the corresponding Layer 3 or Layer 4 write happens immediately. The dashboard reflects the change within seconds.
- Per-session-end. Session-level rollups (total tokens, total cost, flag count, ended_at) write at session end. Hand-offs that batched during the session are emitted to their downstream systems.

HubSpot synchronization

Every Layer 6 persistence emits a HubSpot custom event for the member's contact record. This keeps the relationship spine current without requiring HubSpot to hold the full conversation record. Event types:

- `g4l_idq_completed` — new IDQ retake, with ID Score and band
- `g4l_pulse_completed` — new Pulse Check submission
- `g4l_flagship_completed` — gating asset completion, with flagship code
- `g4l_drift_event` — drift detected; mirrored when recovered
- `g4l_phase_transition` — phase change, with from/to phases
- `g4l_handoff_988` — crisis routing (anonymized in HubSpot to a counter; full record in warehouse only)

Lifecycle stage transitions (e.g., Member → Cycle Complete) flow through the Founder Agent's review queue before they land on the HubSpot record. The Member Agent does not change lifecycle stage directly.

Hand-off mechanisms

Hand-offs are emitted as rows in `conversation_handoff` with payload describing the request. Downstream systems pick up via:

- Founder Agent draft request — Supabase Realtime subscription on the Founder Agent service; draft generation kicks off; draft lands in Jay's review queue in HubSpot
- Coach notification — webhook to the coaching application; pages the assigned coach
- 988 routing — does NOT emit an external call. The agent's in-conversation response contains the 988 number and the warm acknowledgment. The hand-off row is for internal escalation: Donna or the on-call human reviews flagged conversations within 24 hours per the AI Governance Framework.
- Community routing — Circle space invitation or a community thread suggestion; surfaces in the dashboard's Do This Next CTA

Voice and governance enforcement at this layer

Layer 6 is the audit-trail boundary. Every conversation is reviewable. The AI Governance Framework's data-governance commitments (retention period, deletion on request, no model training on member data) are enforced at the warehouse via row-level security and a separate retention policy job.

Voice drift detection runs against the conversation_turn table: a sampling job (Section 5.5) pulls random agent turns and compares them against Brand Standards exemplars. Drift detection raises a `conversation_flag` row with `flag_type = 'voice_drift'`.

Failure modes

Mid-session API failure leaves orphaned turn rows. Mitigation: each turn write is idempotent; if the model call fails, the member-facing message describes a temporary issue and the session continues on retry. Orphaned turns are reconciled by a nightly job that closes sessions whose `ended_at` is null and whose last turn is over an hour stale.

HubSpot event delivery failure. Custom events to HubSpot can fail (network blips, rate limits). The emit is queued via a Supabase function; failed emits retry with exponential backoff; persistent failures alert the operations channel.

Downstream system unavailable for hand-off. Founder Agent or coaching app may be down. Hand-off rows queue; downstream picks up when available. Crisis 988 routing is never blocked — the member-facing response always contains the 988 number even if the internal escalation hand-off fails to emit.

Dependencies

Layer 6 depends on the conversation surface (chat.grintaforlife.com) for transcript capture. Depends on the Founder Agent service for draft hand-offs. Depends on HubSpot's custom event API. Tom Miller dependency: the hand-off types and triggers reflect Tom v4.2 — the Day 105 False Start hand-off pattern, the IDQ intake welcome hand-off pattern, the Day 22 community peer-mirror hand-off pattern. If Tom v4.x adds new hand-off triggers, the `conversation_handoff` table's `handoff_type` enumeration and the trigger logic update.

4. The Four Operating Moments

Every Member Agent conversation falls into exactly one of four operating moments. Each moment has a defined system prompt slice, retrieval strategy, expected outputs, and hand-off pattern. The conversation surface routes the member to one of the four at session start; the agent's behavior shapes accordingly.

4.1 Onboarding

Trigger

First conversation after Member application and AI consent. Exactly once per member per cycle (Cycle 1 at first onboarding; Cycle 2 onboarding at Loop onset with a different prompt slice — see Section 4.5).

Conversation structure

Four chapters, moving the member from their athletic past to their identity synthesis. The agent paces; the member sets the depth.

- Chapter 1 — The athletic past. What kind of athlete or active person were you, before? Sports, training, the body you remember having.
- Chapter 2 — The gap. What happened? The Door that opened (or several). The specific moment you noticed you were no longer that person.
- Chapter 3 — Right now. Where you are this week. What you can and can't do. The honest current state, without polish.
- Chapter 4 — Identity synthesis. What you want back. The version of yourself you can imagine reclaiming. The Reclaim List in seed form.

Outputs

- Member identity paragraph — narrative synthesis in G4L voice that becomes the hero centerpiece of the dashboard. Pairs with the phase-dynamic verb ("Reconnecting: THE ATHLETE"). Stored in `member_profile.identity_paragraph`.
- Structured data object — `intake_athletic_past`, `intake_gap`, `intake_right_now`, `reclaim_list_seed` (7 items), `named_door` (one of 8). All written to `member_profile`.
- No IDQ administered here. IDQ intake runs as a separate session (4.2) after onboarding completes.

System prompt slice (Onboarding)

```
OPERATING MOMENT: Onboarding (Cycle ${cycle_indicator})
YOUR TASK
You are conducting the structured intake conversation with a new
Member. Move through four chapters at the member's pace.
Listen first. Reflect what they say. Ask one question at a time.
CHAPTER SEQUENCE
1. The athletic past
2. The gap
3. Right now
4. Identity synthesis
OUTPUTS YOU WILL PRODUCE
- A member identity paragraph (G4L voice, 3-5 sentences)
- A proposed identity noun (one word or short compound noun)
- A Reclaim List seed (7 items, member-stated)
- The member's named Door (one of the 8)
IDENTITY SYNTHESIS PATTERN
- Synthesize from the member's own words.
- Propose the noun. Example: "It sounds like THE ATHLETE."
- Ask the member to confirm or edit.
- Commit only on explicit confirmation.
DO NOT
- Conduct the IDQ here. That runs in a separate session.
- Suggest a tier or commercial offering.
- Diagnose or label the member's experience.
CLOSING
When the four chapters are complete and the noun is confirmed,
preview: "Your IDQ will be next. Jay reviews each welcome
personally and will reach you within 24 hours."
```

Hand-offs at session end

- Founder Agent welcome request — Founder Agent drafts the personal welcome email from Jay; lands in Jay's review queue
- Dashboard update — hero centerpiece populates with the confirmed identity noun
- Next-session prompt — member receives the IDQ intake invitation

Tom Miller dependency

Onboarding conversation structure (four chapters) is from Tom Miller v4.2 Section D.2 and the Identity Synthesis pattern. The four-chapter sequence is locked at v4.2. Future Tom Miller iterations may add chapters or restructure; the system prompt slice and the structured data object schema update accordingly.

4.2 IDQ Retake

Trigger

Every 60 days from baseline. The conversation surface surfaces an IDQ Retake invitation when `days_since_last_idq >= 60`. Members can skip; the framework respects the Independence Guarantee. A skipped retake produces a `behavioral_signal` row with `signal_type = 'idq_skipped'`; the next 60-day window opens after the skip.

Conversation structure

24 items across 4 IDQ dimensions (6 items per dimension). The agent administers conversationally — one item at a time, with brief framing — rather than as a form. The conversational mode preserves engagement past the second retake; form-based retakes drop off after Cycle 1.

Outputs

- 24-item response set — Likert 1–5 each, stored in `idq_retake.responses` jsonb
- Sub-scale scores — Physical, Identity, Relational, Forward; 0–25 each
- Composite ID Score — 0–100, computed as `(total_raw / 120) * 100`
- Band label — Starting Line / The Fade / Drifting / In Flow
- Delta from baseline and delta from previous retake — both stored on the retake row
- In-conversation reflection — warm, names what just happened. Does not reveal the ID Score as a raw number without the band label and plain-language interpretation.

System prompt slice (IDQ Retake)

```
OPERATING MOMENT: IDQ Retake #${sequence_no} (Cycle ${cycle_indicator})
YOUR TASK
Administer the 24-item Identity Distance Questionnaire
conversationally. One item at a time. Brief framing per dimension.
ITEM PRESENTATION
- Present the item stem and the 1-5 Likert anchors.
- Accept member's response.
- Move to next item without commentary.
- After every 6 items (one dimension complete), brief one-sentence
  transition: "That's Physical. Moving to Identity now."
AT COMPLETION
- The runtime computes ID Score, dimension scores, band, deltas.
- You receive the computed values as a SCORE_RESULT message.
- Produce a 2-3 sentence reflection in G4L voice:
   * Acknowledge what just happened (warm, member-paced)
   * Name the band label (NOT the raw number)
   * Name one dimension that moved (positive direction preferred)
   * Preview: "Jay will see this and may reach out within 24 hours."
DO NOT
- Reveal the ID Score as a raw number in your reflection.
  The dashboard shows the number with the band label and
  interpretation. Your reflection names the band, not the digit.
- Interpret a low score as failure. The framework treats Starting
  Line as honest, not bad. 36 is honest; 70 is meaningful; 90 is
  well-earned.
- Promote tier upgrades or commercial offerings.
```

Hand-offs at session end

- Dashboard update — ID Score, dimensions, trajectory, band, plain-language interpretation
- Founder Agent retake commentary request — Founder Agent drafts Jay-voice retake commentary; lands in review queue
- SMS notification (if opted in) — "Your IDQ just landed. Score on your dashboard."
- Research feed write — anonymized retake row to the IRB research view

Tom Miller dependency

The 60-day cadence, the four IDQ dimensions with 6 items each, the four named bands, and the in-conversation reflection pattern are from Tom Miller v4.2 Section D.3 (Cycle 1 Trajectory). Tom v4.x updates to instrument items or cadence cascade into the `idq_retake` schema and the agent's administration logic.

4.3 Pulse Check

Trigger

Monthly between IDQ retakes. Lower friction than the full IDQ — 3 to 5 questions rotating IDQ dimension coverage. The conversation surface surfaces the Pulse when `days_since_last_pulse >= 30` and `days_since_last_idq < 60`.

Conversation structure

Brief. 3–5 minutes. The agent presents items conversationally, accepts responses, computes a signal strength (0–100) based on response patterns, and produces a one-sentence reflection.

Outputs

- Item responses — 3-5 items, stored in `pulse_check.responses` jsonb
- Sub-scale focus — which the IDQ dimensions rotated in this month
- Signal strength — 0–100, agent-computed; surfaces patterns the framework should attend to between IDQ retakes
- No band update. Pulse Check does not change the ID Score. The dashboard shows the most recent IDQ as the canonical score; the Pulse Check signal surfaces as a small indicator.

Tom Miller dependency

Pulse Check cadence (12 Pulses per Cycle alongside 6 IDQ retakes) is from Tom v4.2 Section D.3. Items rotate the IDQ dimensions coverage; the item bank is governed by the IDQ instrument design (Layer 2). Tom v4.x updates to Pulse cadence or item bank cascade here.

4.4 Ongoing Check-in

Trigger

The agent's default mode when no other operating moment is scheduled. Member can open the conversation surface at any time and start a check-in conversation. The agent draws on the member's full history to make each conversation feel continuous.

Conversation structure

Member-led. The agent opens warmly, references the most recent event the member experienced (a gating asset completion, a False Start, a community moment), asks one question, listens. The conversation goes wherever the member takes it. The agent reflects, ranges across Layer 4 history as needed, and routes when appropriate (to community spaces, to Greg's Concept Map for science questions, to Direct tier coaches for coaching questions, to the Founder Agent for moments worth Jay's personal touch).

Outputs

- Conversation transcript (Layer 6)
- Optional profile refinement — if the member updates their identity paragraph, identity noun (Cycle 2 only), or Reclaim List during the conversation
- Behavioral signal if a meaningful event surfaces (a Window completion the member just did, a False Start the member is in the middle of)
- Hand-offs as appropriate — community routing, coach notify, Founder Agent draft request

System prompt slice (Ongoing Check-in)

```
OPERATING MOMENT: Ongoing Check-in
YOUR TASK
Be present. The member came to check in. Open warmly. Reference
the most recent meaningful event from their Layer 4 history.
Ask one question. Listen.
OPENING PATTERNS
- After a gating asset completion: "I see you finished [gating asset].
  How did it land?"
- After a False Start log: "Day [N] was a False Start. Have you
  re-entered?"
- After a long gap: "It's been [X] days. What's on your mind today?"
- After a milestone (100 miles, 500 miles, etc.): "That's
  [milestone]. What's that feeling like?"
DURING THE CONVERSATION
- Range across the member's history as needed. Reference specific
  events, not vague generalities.
- Reflect what they say. Don't redirect.
- Route when appropriate:
    * Science question → Concept Map / Science Check
    * Coaching question (Direct tier) → coach notify hand-off
    * Community question → Circle space routing
    * Moment worth Jay's touch → Founder Agent draft request
- One question at a time. Never two. Never three.
DO NOT
- Push the member toward a specific action they didn't ask for.
- Reveal the ID Score without band and interpretation.
- Diagnose. Reflect.
```

Tom Miller dependency

Ongoing Check-in's pattern of referencing recent events and ranging across history is from Tom v4.2 Section D.4 — the False Start re-entry conversation on Day 105, the Adventure Plan posting on Day 280, the Cycle 2 onset conversation on Day 18 after the stroke. Tom v4.x adds or refines patterns; the system prompt slice updates.

4.5 Cycle 2 onboarding — the Loop

Trigger

The Cycle 2 onboarding fires when a new Door opens and the member returns to Reconnect. The conversation surface detects: an extended absence (>14 days) followed by a return; or an explicit "new Door opened" signal from the member.

Cycle 2 differs from Cycle 1 onboarding in three places

- *Identity Check-In three paths.* The agent does not assume the prior identity noun carries forward. It asks. Three explicit paths: keep the prior noun, hold a compound (e.g., THE ATHLETE-SON), adopt a new noun. The framework supports all three; the member chooses.
- *Member memory carries forward.* The Visualization Workshop anchor statements from Cycle 1, the Fuel Plan built in Cycle 1, the False Start Protocol the member has actually used — all remain available. The agent acknowledges this in the conversation: "You have the anchors from last year. You have the Fuel Plan you built. None of that goes away."
- *Cycle indicator increments.* `cycle_indicator = 2`. All Layer 4 writes from this session forward carry the new cycle.

Tom Miller dependency

The Cycle 2 onboarding is the most Tom Miller-dependent operating moment. The three-path Identity Check-In is from Tom v4.2 Section D.5 — Tom adopting THE SON after his father's stroke. The framework's recursive design is articulated through Tom's specific example. Tom v4.x is the canonical reference for Cycle 2 patterns; this Spec section moves with Tom.

5. Voice and Governance Enforcement

The Member Agent's voice fidelity and governance compliance are not aspirations — they are enforced architecturally. This section describes the enforcement surfaces, the monitoring cadence, and the regression patterns that keep the agent honest over time.

5.1 System prompt structure

The system prompt is assembled at conversation start by the agent runtime. It has four parts, concatenated in order:

- Base prompt (Layer 1). Constant across all conversations. Voice rules, prohibitions, reflect-and-route patterns, AI disclosure, Independence Guarantee.
- Program knowledge slice (Layer 2 — always-loaded core). Compact summaries of the 4Rs, 8 Doors, Atlas asset codes, IDQ structure, ID Score bands, Identity Synthesis pattern.
- Operating moment slice (Section 4). One of four — Onboarding, IDQ Retake, Pulse Check, Ongoing Check-in. Defines the conversation structure for the session.
- Member context (Layers 3 + 4 rollup + Layer 5). Compact summary of who the member is, where they are in the framework, what the session context is.

Total system prompt size: typically 4,000–6,000 tokens. The Anthropic models handle this without truncation.

RAG-retrieved content (deep Layer 2 entries, specific Topical Syntheses) is injected per-turn as user-side context messages, not in the system prompt. This keeps the system prompt small enough to deploy quickly when voice rules update.

5.2 Voice rules enforcement

Voice rules are enforced through three mechanisms in increasing order of strictness:

Mechanism 1 — Prompt specification

The system prompt's base section declares the voice rules in plain language. The model's instruction-following keeps most outputs compliant most of the time. This is the first line of defense.

Mechanism 2 — Locked vocabulary capitalization

Brand Standards v1.2 locks capitalization for defined vocabulary (The Fade, Identity Distance, the 4Rs, ID Score, etc.). The agent's system prompt includes the locked vocabulary list with a directive: "Locked vocabulary capitalized on first mention each conversation." Voice drift on capitalization is a leading indicator of broader drift.

Mechanism 3 — Forbidden phrase regex

Specific forbidden phrases are caught by a regex check on agent outputs before the model's response is returned to the member. The check is non-blocking: a hit logs a `conversation_flag` with `flag_type = 'voice_drift'` and re-prompts the model with a corrective instruction. Forbidden phrases (from Brand Standards):

```
// /lib/agent/voice-check.ts
const FORBIDDEN_PATTERNS: RegExp[] = [
  /\bjourney\b/i,                // The word "journey"
  /\bI hear you\b/i,             // The empathy filler
  /\bThat makes sense\b/i,       // The acknowledgment filler
  /\bamazing\b/i,                // Wellness-app filler
  /\bawesome\b/i,                // Wellness-app filler
  /\bit's not [^,.]+, it's\b/i,  // Negate-then-substitute pattern
];
export function checkVoice(text: string): VoiceCheckResult {
  const hits = FORBIDDEN_PATTERNS.filter(p => p.test(text));
  return { passed: hits.length === 0, hits };
}
```

When a check fails, the runtime emits a `voice_drift` flag and re-prompts the model: "Your previous response contained [phrase]. Revise without it." The re-prompted response is the one the member sees.

5.3 Governance prohibition enforcement

The AI Governance Framework prohibitions (Layer 1) are enforced through the same three-mechanism stack as voice rules, with one critical addition: the 988 protocol is enforced through pattern detection on member turns, not only on agent turns.

988 protocol implementation

The runtime scans incoming member turns for distress patterns before passing to the model. Detection triggers an immediate protocol shift:

```
// /lib/agent/crisis-detection.ts
const CRISIS_PATTERNS: RegExp[ ] = [
  /\b(suicide|kill myself|end (it|my life))\b/i,
  /\b(self[ - ]harm|hurt myself)\b/i,
  /\b(don'?t want to (live|be here|exist))\b/i,
  /\b(no (reason|point) (to live|in living))\b/i,
  // ... full list maintained as production code, reviewed quarterly
];
export function detectCrisis(memberTurn: string): CrisisDetection {
  const hits = CRISIS_PATTERNS.filter(p => p.test(memberTurn));
  if (hits.length === 0) return { detected: false };
  return {
    detected: true,
    patterns: hits.map(p => p.source),
    protocol_activated_at: new Date(),
  };
}
```

When a crisis pattern detects, the runtime:

1. Prepends a CRISIS_PROTOCOL_ACTIVE marker to the model's context window for this turn and all subsequent turns in the session
2. Injects the 988 protocol response template into the model's instructions: warm acknowledgment, no counsel attempt, 988 phone number with format "call or text 988 for the Suicide and Crisis Lifeline"
3. Writes a `conversation_flag` row with `flag_type = 'crisis_signal'`
4. Emits a `conversation_handoff` row with `handoff_type = '988_route'` — this is for internal escalation review (Donna or on-call human within 24 hours), not for an external call

The member's experience is: their disclosure is met warmly, the 988 number is offered, and the agent does not press the topic further. The escalation hand-off is invisible to the member.

Member retains full control of the conversation. The agent does not end the session unilaterally. If the member redirects to a different topic, the agent follows.

Other governance prohibitions

- No raw ID Score reveal — enforced by IDQ Retake operating moment system prompt slice; the runtime also post-filters agent outputs to verify a band label accompanies any number in the 0–100 range
- No tier upgrade suggestions — enforced by prompt specification and a regex check for upgrade language ("upgrade," "premium," "unlock")
- No coaching substitution at Direct tier — enforced by the Ongoing Check-in slice; the agent routes coaching questions to the coach notify hand-off rather than answering
- No operation outside G4L voice and 4R framework — primarily a voice rules concern; the locked vocabulary and forbidden phrase checks catch most departures
- No persistence beyond explicit consent — enforced at the Supabase row-level security layer; the agent cannot read fields the member has not consented to share
- No continuation after a stop signal — runtime detects stop signals ("stop," "I'd like to end this," explicit "end conversation" UI action) and closes the session immediately; the agent acknowledges and the session ends

5.4 Provider abstraction layer

The agent runtime wraps the Anthropic API in a provider-agnostic interface. This is one of the six Consistency and Reliability Commitments and the hedge against vendor lock-in noted in Provider Selection Rationale (Section 2.3).

```
// /lib/agent/providers/interface.ts
export interface LlmProvider {
  generate(opts: {
    systemPrompt: string;
    messages: Message[ ];
    tools?: ToolDefinition[ ];
    maxTokens?: number;
    temperature?: number;
  }): Promise<GenerationResult>;
  streamGenerate(opts: ...): AsyncIterable<GenerationChunk>;
  countTokens(text: string): Promise<number>;
}
// Implementations:
// - /lib/agent/providers/anthropic.ts (primary)
```

```
// - /lib/agent/providers/openai.ts    (fallback / evaluation)
```

The abstraction is shallow by design — the interface mirrors common LLM API surface, not the lowest-common-denominator. Provider-specific features (Anthropic's tool use, prompt caching) are accessible via optional fields. Switching providers is a configuration change and a regression-test pass, not a rewrite.

The 90–180 day post-launch evaluation (Provider Selection Rationale, Appendix C v1.3) runs through the abstraction layer with both Anthropic and an alternative provider receiving the same prompts. Voice fidelity, refusal behavior, and cost are measured comparatively.

5.5 Voice drift monitor

The voice drift monitor runs on a 90-day cadence per the AI Governance Framework's review cadence. The procedure:

1. Sample 20 random agent turns from the past 90 days, stratified across the four operating moments
2. Compare each turn against Brand Standards voice exemplars using both an automated check (forbidden phrase regex, locked vocabulary capitalization rate, sentence length distribution) and a human review (Donna + Jay)
3. Score each turn on a five-point voice fidelity scale — anchored against locked exemplars
4. Aggregate the scores; if average drops below threshold, trigger system prompt recalibration
5. Drift hotspots (specific operating moments or topic areas with consistent drift) inform targeted prompt updates

The monitor's first run is scheduled 90 days after October platform launch. Earlier monitoring during pre-launch runs as a manual review of Jay-direct correspondence drafted by humans against the same standards.

Locked exemplars

Voice exemplars are a small (10–20) corpus of agent turns reviewed and approved as canonical for each operating moment. They live in `program_knowledge` with `doc_type = 'voice_exemplar'`. New exemplars are added when the team encounters a turn that lands particularly well; old exemplars are retired when Brand Standards evolves.

The exemplars serve three purposes: human review reference for drift detection, system prompt few-shot material if needed for specific operating moments, and the standard against which voice fidelity is measured over time.

6. Failure Modes and Resilience

Architecture honesty means naming what can fail. This section names the failure modes the design anticipates, the mitigations in place, and the boundary between resilient behaviors and member-visible degradation.

6.1 Provider API failure

The Anthropic API can fail — transient network issues, rate limit hits, model availability windows, or larger outages. Failure modes by severity:

- Single-call transient failure (network blip, 503 error) — automatic retry with exponential backoff (3 attempts, 500ms / 2s / 8s). Most resolve within the retry window; member never notices.
- Rate limit hit — request queued; member sees a brief "thinking..." indicator; processed within the queue's drain window. If queue depth exceeds threshold, falls back to degraded mode (6.2).
- Sustained provider outage (>5 minutes) — graceful degradation activates. Member-facing message: "The agent is having a moment. Your IDQ and gating asset submissions are still saving. We'll be back shortly." Session pauses; existing data persists; member can return when service restores.
- Provider failover — if the alternative provider is available and the abstraction layer's failover policy is set, the system can switch to the backup provider. Voice fidelity is verified against exemplars before failover is permitted; the team confirms the alternative passes regression.

6.2 Graceful degradation

When the agent runtime cannot generate a response — provider outage, retrieval failure, or any other architectural fault — the member experience degrades gracefully without losing data.

Member-facing

- Chat surface shows a status banner: "The agent is offline temporarily. Your dashboard and the Atlas are still available."
- In-progress IDQ submission saves locally and to the warehouse before the failure surfaces; the member can resume when service returns
- Founder Agent drafts already queued continue to land in Jay's review queue independent of Member Agent availability
- Tovuti gating asset views remain accessible; the framework operates on paper protocol if needed (Section 6.3)

Internal

- Alerting fires to the operations channel on the first sustained failure
- Status page updates if outage exceeds 10 minutes
- On-call rotation handles escalation; Donna or Jay informed within 30 minutes for member-impacting outages

6.3 Independence Guarantee paper protocol

The Independence Guarantee is the framework's commitment that no member is blocked from the work because technology fails them. The paper protocol is the operational expression of that commitment.

Every gating asset has a printable paper workflow that produces the same structured output the digital experience produces. The workflows live in `program_knowledge` with `doc_type = 'paper_protocol'` and are downloadable from the dashboard's Independence Guarantee Fallback link.

- R-1 IDQ paper form — 24-item printable; member completes; staff transcribes into the warehouse; ID Score computed; band assigned
- R-4 Identity Excavation worksheet — five-evening guided exercise on paper; 7-item Reclaim List captured manually
- R-6 Window Exercise tracker — printable commitment card and weekly tracking sheet
- W-1 Disinformation Audit log — printable 48-hour input log
- W-3 Visualization Workshop guide — printable audio script and anchor-statement capture form
- W-5 False Start Protocol card — laminated wallet-size five-step prompt
- B-1 First Step Assessment — printable four-domain reflection
- B-3 First 1,000 Miles tracker — printable mile log
- B-5 Fuel Plan worksheet — five-component reflection and capture
- C-1 Reclaim Readiness Assessment — 12-item printable
- C-3 Adventure Planning Worksheet — five-section printable
- C-5 Your Success Story — five-act narrative prompts and writing space

Paper completions are entered into the warehouse manually by staff or the member. The framework treats paper and digital completions identically for cycle progression and impact measurement. The Member Agent never gatekeeps.

6.4 Account redundancy

Two Anthropic API credentials, primary and failover. The failover credential is held by Donna; the primary by Jay/operations. Rotation policy: annual rotation; immediate rotation if a credential is suspected compromised. Both credentials are valid simultaneously so failover is instantaneous and does not require a new credential issue cycle.

Credentials are stored in Supabase Vault (encrypted at rest, accessed via service role). They never appear in application code, logs, or version control.

6.5 Cost ceiling

Per-conversation token budget: 50,000 tokens default (configurable). Most conversations land well under (typical Ongoing Check-in: 8,000 tokens; typical IDQ Retake: 15,000 tokens including all 24 items). The budget protects against runaway loops.

Monthly aggregate cap: set per the operating budget. Alarms at 75% and 90% of monthly cap. Hard cap triggers non-essential operation pause:

- Essential (always runs): member-facing conversations across all four operating moments
- Non-essential (paused at hard cap): Founder Agent social copy generation, batch retrospective analysis, voice drift sampling beyond minimum cadence

Hard cap reached → alerting; manual budget review; budget increase or essential-only operation for the remainder of the cycle. Member-facing operations are never blocked by cost ceiling.

6.6 Audit trail

Every conversation produces a complete audit trail in the warehouse — system prompt version, model version, every turn, every tool call, every hand-off emitted, every flag raised. The trail is queryable and exportable for:

- AI Governance Framework quarterly review
- Foundation reporting on safety incidents
- Member data export on request (GDPR-compatible)
- Research IRB compliance verification
- Incident post-mortem if a member raises a concern

The audit trail is retention-policy controlled: 5-year retention for conversation_turn and conversation_session; indefinite for conversation_handoff and conversation_flag. Members can request deletion of their own conversation data; the AI Governance Framework's retention and deletion commitments are enforced via a scheduled retention job.

7. Build Sequence

The Member Agent build targets a single platform launch in October 2026 with the full Member Agent live from day one. This section maps the agent build into Platform Development Plan v4's phase sequencing and names the MVP boundary.

Note on launch model. Per the May 2026 team brainstorm, the Two-Stage Launch (pre-launch June–October + platform launch Platform mid-October onward) is retired. Single platform launch in October 2026. The second edition of the book delivers publicly in June 2026 as a separate marketing artifact, independent of platform enrollment. Pre-launch marketing through podcast, screenings, social, newsletter, and referral channels runs through summer 2026.

7.1 Pre-launch build — May through mid-October 2026

The Member Agent is built and tested through summer 2026 against synthetic conversations and a small founding-cohort dry-run. No public-facing member operations until October launch.

- Layer 1 (Agent Identity) — system prompt drafted, reviewed, locked. Voice exemplars assembled. Forbidden phrase regex catalogued. 988 protocol patterns drafted and reviewed by Donna.
- Layer 2 (Program Knowledge) — core program corpus seeded from BP v6.1 Section IV-A-2 and Brand Standards v1.2/v1.3. Greg's Concept Map drafted by end of July (per Authoring Brief v1.2). Initial Science Checks layering in across August–September.
- Layer 3 (Member Profile) — schema deployed and tested with synthetic Members and a small dry-run cohort. Schema includes the renamed columns (`self_score`, `social_score`, `outlook_score`, `id_score`). Drops the `band` field per the May 2026 cascade.
- Layer 4 (Member History) — schema deployed. `idq_retake` table uses new column names; `pulse_check` table replaces the old `monthly_pulse` table. Founder Agent draft pipeline operates in shadow mode through summer, drafting against synthetic data and reviewed against Jay's own writing.
- Layer 5 (Conversation Context) — runtime built and tested against synthetic conversations; goes live at October launch.
- Layer 6 (Conversation Output) — schema deployed; conversation_session, conversation_turn, conversation_handoff, conversation_flag tables created; HubSpot custom event integration tested with synthetic events.

7.2 October platform launch — MVP scope

October platform launch is the moment the Member Agent goes live to Members. The MVP scope is deliberate — what must work on day one and what can layer in across Q4 2026.

In MVP scope (must ship at October platform launch)

- All four operating moments operational — Onboarding for new Members; IDQ intake on Day 1 + IDQ retake conversationally every 60 days; Pulse Check every 30 days between retakes; Ongoing Check-in always available
- Layer 1 + Layer 2 core program corpus live — system prompt deployed, locked vocabulary enforced (Member, asset, ID Score, Pulse Check, four IDQ dimensions: Physical/Self/Social/Outlook), 988 protocol active, AI disclosure first-line script live, bands retired
- Greg's Concept Map live in Layer 2 — at least the four Phase-level sections complete by October launch; remaining Layer / gating-asset-level sections may layer in across November
- At least 6 of 12 Science Checks live — Reconnect (R-1, R-4, R-6) and Rebuild (B-1, B-3, B-5) prioritized as the highest-leverage gating assets for the founding cohort
- Provider abstraction layer in place — Anthropic API primary; abstraction wraps the calls so a future provider switch is configuration, not code rewrite
- Graceful degradation tested — provider outage simulation passes; member-facing degradation message rehearsed; paper protocol downloadable links live on the dashboard's Independence Guarantee Fallback section
- Audit trail operational — conversation_session / conversation_turn / conversation_handoff / conversation_flag write paths exercised end-to-end
- HubSpot custom event integration live — all six event types emit successfully on representative test conversations
- Voice drift monitor scheduled — first run 90 days after launch (mid-January 2027)

Layers in after launch (Q4 2026 and beyond)

- Remaining Science Checks (Rewire and Reclaim — W-1, W-3, W-5, C-1, C-3, C-5) — drafted by Greg through October–November per Authoring Brief; Layer 2 deployment continues post-launch
- Topical Syntheses — drafted by Greg through October–November; deployment as drafted and reviewed
- Cycle 2 onboarding operating moment — first Members reach Cycle 2 in mid-2027; the operating moment is implemented at launch but only exercised when the first Cycle 2 Loop opens
- Founder Agent full set of six MVP jobs — Founder Agent's six jobs (welcome, drift, milestone, retake commentary, Monday briefing, social copy) covered in Appendix C v1.3 Section C.5; agent build runs in parallel with Member Agent build

- Second edition Foreword (Greg) — drafted Q1 2027 alongside second-edition production cycle
- Operating moments diagram (companion to Data Model SVG) — built; companion artifact to the Data Model SVG

7.3 Mapping to Platform Development Plan v4

This Spec aligns with Platform Development Plan v4's three-phase build sequence, now collapsed under the single October launch model:

PDP v4 Phase	Member Agent build activity	Calendar alignment
Phase 1: Foundations (May–July 2026)	Layer 3 + Layer 4 schemas deployed; warehouse + HubSpot integration; system prompt drafted; voice exemplars assembled	Pre-launch foundations
Phase 2: Differentiate (August–September 2026)	Layer 1 + Layer 2 + agent runtime built; operating moments implemented; Greg's Concept Map ingested; provider abstraction layer; voice and governance enforcement	Pre-launch differentiation
Phase 3: Customize (October 2026)	End-to-end testing; voice drift baseline established; graceful degradation tested; remaining Science Checks ingested; AI Governance Framework PDF published on g4l.org	Launch mid-October 2026

Phase numbering refers to the platform build calendar weeks, not the Member experience phases (Reconnect / Rewire / Rebuild / Reclaim). Member phase boundaries are gated by gating asset completion, not by calendar weeks.

7.4 Developer selection criteria

The Spec assumes the agent build is led by a developer or developer team with the following capabilities:

- Production-grade TypeScript / Next.js experience — the reference architecture is Next.js on Vercel; comfort with App Router, Server Components, Server Actions, Edge Functions, and Vercel deployment is required
- LLM application experience — at least one prior agent or RAG application in production; understanding of system prompt design, RAG retrieval patterns, and conversation state management
- Supabase / Postgres experience — schema design, row-level security, pgvector for embeddings, Supabase Auth and Realtime
- Voice and safety sensitivity — willingness to read Brand Standards v1.2/v1.3 and AI Governance Framework v1.2; commitment to voice fidelity and governance enforcement as architectural requirements, not afterthoughts
- Iteration discipline — comfort with version-controlled assumptions and Tom Miller dependency markers; willingness to ship for iteration rather than for finality

The Spec is designed to be readable by a developer with these capabilities; sections of the Spec may be opaque to developers without LLM application experience. An onboarding session walking the developer through this Spec and the Member Agent Data Model SVG is recommended before the first ticket.

8. Open Decisions

The Spec names many decisions and proposes defaults for each. The list below collects the open decisions where the proposed default may benefit from a technical co-pilot's review, an agency's counter-proposal, or a developer's empirical testing. Each has a proposed default, the trade-off, and the recommended decision moment.

8.1 RAG retrieval Top-K

Default: K = 3 per retrieval call.

Trade-off: Higher K increases retrieval coverage (less likelihood of missing relevant content) but increases context window cost and may dilute the model's attention. Lower K cuts cost but increases miss risk.

Decision moment: Empirical testing during Phase 2 build. Sample 50 conversations across operating moments; measure retrieval relevance at K = 1, 3, 5, 7. Lock the default that maximizes retrieval relevance × cost ratio.

8.2 Embedding model selection

Default: OpenAI text-embedding-3-small (1536 dimensions) or Anthropic's embedding model when available.

Trade-off: OpenAI embeddings are mature, well-supported by pgvector, and inexpensive. Anthropic embeddings (if available at build time) would simplify the provider story (same vendor for generation and embeddings). The 1536-dimension schema is forward-compatible with most embedding models.

Decision moment: During Phase 2 build. Re-embedding when a new model emerges costs runtime but does not block. Schema is forward-compatible.

8.3 Conversation memory window strategy

Default: Full Layer 3 + Layer 4 rollup loaded at conversation start; recent events (last 30 days) verbatim; deep retrieval on demand.

Trade-off: Larger window improves continuity but increases per-conversation cost. Shorter window cuts cost but may make conversations feel more transactional.

Decision moment: First voice drift review (90 days post-launch) — if conversations feel transactional or repetitive in the review, widen the window.

8.4 Per-conversation token budget

Default: 50,000 tokens per conversation.

Trade-off: Higher budget prevents accidental cut-offs but increases worst-case cost. Lower budget controls cost but risks awkward truncation.

Decision moment: During Phase 3 testing. Observe actual usage distribution; set budget at 95th percentile of observed conversations.

8.5 Founder Agent — Member Agent boundary

Default: Member Agent never drafts in Jay's voice; Founder Agent never speaks in real-time to members. Two distinct agents with shared corpus but separate system prompts and separate operational responsibilities.

Trade-off: Clean architectural separation; some duplication of context loading. An alternative architecture would unify the two agents with mode switching; cleaner code at the cost of harder voice-boundary enforcement.

Decision moment: Locked at Spec v1.0. Re-evaluate if voice drift monitor surfaces consistent boundary confusion.

8.6 SMS vs. in-app notification balance

Default: SMS for four moments only (welcome after onboarding, drift nudge, ID Score notification after IDQ retake, milestone recognition). Other notifications happen in-app via dashboard CTAs.

Trade-off: SMS is intrusive but high-attention; in-app is opt-in but easy to miss. The current default is intentionally conservative — protects member trust.

Decision moment: First member feedback survey (90 days post-launch). Adjust if members request more or fewer SMS touchpoints.

8.7 Dashboard real-time refresh strategy

Default: Supabase Realtime subscription on the member's record; dashboard updates within seconds of any Layer 3 or Layer 4 write.

Trade-off: Realtime adds infrastructure complexity but delivers the member experience the Spec implies. Polling-based refresh would be simpler but feel less alive.

Decision moment: Phase 2 build. Supabase Realtime is part of the reference architecture; deviation requires SOW justification.

8.8 Cycle 2 detection threshold

Default: Cycle 2 is signaled either by the member completing C-7 Cycle Closing and then returning, OR by an explicit "new Door opened" indicator in the member's next session opener.

Trade-off: Automatic detection on absence ("member absent 14+ days, then returned, must be Cycle 2") is wrong — members take breaks without cycling. The explicit indicator is reliable but requires the member to surface it.

Decision moment: First Cycle 2 onset (estimated mid-2027). Pattern emerges from actual Member behavior; refine then.

8.9 pre-launch to platform launch Member data migration

Default: All pre-launch Typeform IDQ responses and Member intake data migrate into Layer 3 and Layer 4 at platform launch cutover. Identity paragraphs from pre-launch (drafted by Jay) become the seed for Member Agent ongoing context; the Member Agent can refine them during the member's first post-cutover Ongoing Check-in if the member consents.

Trade-off: Full migration preserves Member continuity; alternative would be re-onboarding through the Member Agent. Re-onboarding costs Member time and risks loss of the concierge-relationship intimacy.

Decision moment: Locked at Spec v1.0. Migration runbook is part of Phase 3 testing.

8.10 Voice exemplar corpus size

Default: 10–20 locked exemplars per operating moment (40–80 total).

Trade-off: More exemplars cover more variation but require more curation. Fewer exemplars miss edge cases.

Decision moment: First voice drift review (90 days post-launch). Adjust based on drift patterns observed.

9. Tom Miller v4.2 Dependencies

The Spec is built for iteration. Tom Miller Member Journey v4.2 is the canonical member experience example as of this version; the product experience team is iterating Tom. When Tom updates, the Spec sections below update with him. The convention is Tom v4.x → Spec v1.x; Tom v5.x → Spec v2.x.

9.1 Explicit dependency list

Sections of this Spec that depend on Tom Miller v4.2 — when Tom updates, revisit each one:

Spec section	Tom v4.2 reference	Dependency
3.3 Layer 3 — Member Profile	Tom v4.2 §D.1 Tom at Intake	Structured intake chapter schema (athletic past, gap, right now); Reclaim List seed structure
3.3 Layer 3 — Identity Synthesis	Tom v4.2 §D.2 The Opening	Synthesize-propose-confirm pattern; identity noun format
3.4 Layer 4 — gating asset outputs	Tom v4.2 §D.4 Four Phases	Per-gating asset structured output schema (Reclaim List, anchor statements, False Start logs, First Step direction, Adventure Plan, Success Story)
3.4 Layer 4 — Cycle indicator	Tom v4.2 §D.5 The Loop	Cycle 2 onset detection; cycle_indicator increment logic
3.6 Layer 6 — Hand-off triggers	Tom v4.2 §D.4 Days 22, 105, 280	Specific hand-off patterns — community peer-mirror (Day 22), Founder Agent False Start welcome (Day 105), community Adventure Plan sharing (Day 280)
4.1 Onboarding	Tom v4.2 §D.1, §D.2	Four-chapter onboarding conversation structure
4.2 IDQ Retake	Tom v4.2 §D.3 Trajectory	60-day cadence; six retakes per Cycle; in-conversation reflection pattern; band crossing language
4.3 Pulse Check	Tom v4.2 §D.3	Twelve Pulses per Cycle alongside six IDQ retakes; the IDQ dimensions rotation

4.4 Ongoing Check-in	Tom v4.2 §D.4 Day 105, §D.5 Day 18 after stroke	Opening pattern referencing most-recent meaningful event; range across history; one question at a time
4.5 Cycle 2 onboarding	Tom v4.2 §D.5 The Loop	Three-path Identity Check-In (keep / hold compound / adopt new); memory carry-forward; cycle indicator increment

9.2 Versioning convention

Compatibility signal is the version number itself. Anyone reading either document knows the compatibility at a glance.

- Tom Miller v4.x → Spec v1.x. Point-release updates. Content refinement, new examples, minor schema adjustments. Spec updates by editing the dependent sections without restructuring.
- Tom Miller v5.x → Spec v2.x. Major change. Signals an architectural revisit may be warranted — the data model assumptions may need to evolve.

When Tom Miller updates, the procedure is:

1. Read Tom Miller's changelog
2. Walk the dependency table above; mark which sections need revision
3. Update Spec sections; bump Spec version (point release for point Tom release; major release for major Tom release)
4. Notify the developer (or update the SOW) if the build is in flight

9.3 Adjacent dependencies

The Spec also depends on other documents that may update independently of Tom Miller. Listed for completeness:

- Brand Standards v1.2 (v1.3 in progress) — voice rules, locked vocabulary, forbidden phrase list, three Jay registers. Cascade target: Layer 1 system prompt, Layer 2 program_knowledge entries, voice exemplars.
- AI Governance Framework v1.2 — prohibitions, emotional safety protocol, data governance commitments, review cadence. Cascade target: Layer 1 system prompt; Section 5.3 988 protocol; Section 6 retention policy.
- Appendix C v1.3 (Platform Architecture) — reference architecture, subdomain map, eight functional tools, two data spines. Cascade target: Section 2 Reference Architecture; Section 7 Build Sequence.
- Appendix E v1.2 (The Atlas) — 28 assets with codes, phases, layers, purposes. Cascade target: Layer 2 program_knowledge entries; Layer 4 gating asset completion schema.

- G4L × Greg Welk Authoring Brief v1.0 — Greg's three sub-corpora (Concept Map, Science Checks, Topical Syntheses). Cascade target: Layer 2 Greg sub-corpora identifiers and content.
- Platform Development Plan v4 — phase sequencing, Consistency and Reliability Commitments, single platform launch (October 2026) operating model. Cascade target: Section 2.4 reliability commitments; Section 7 Build Sequence.

Each adjacent dependency has its own versioning convention. The Spec version bumps when any dependency change cascades into a Spec-level revision.

10. Glossary and References

10.1 Locked vocabulary used in this Spec

All Brand Standards v1.2 locked vocabulary applies. Quick reference for terms heavily used here:

- The 4Rs — the framework name. Reconnect, Rewire, Rebuild, Reclaim. Always capitalized.
- The Atlas — the named curriculum of 28 assets. Always capitalized.
- The 8 Doors — the eight identity-shifting moments. Always capitalized as a framework; each individual Door capitalized when named (The Career, The Aging Parents, etc.).
- Identity Distance / IDQ / ID Score — the construct, the instrument, the score. Three-layer naming.
- Member — the founding cohort role. Always capitalized.
- Member Agent / Founder Agent — the two AI components. Capitalized.
- The Loop — the framework's recursive design. Always capitalized.
- Independence Guarantee — the commitment that the framework works without technology. Always capitalized.
- Identity Synthesis (Option A) — the synthesize-propose-confirm pattern. Capitalized when named.
- Operating Architecture — the five strategic dimensions governing technology choices. Always capitalized.

10.2 Companion documents (locked corpus)

- Member Agent Data Model — SVG diagram, May 2026 — the front-page reference for this Spec
- Appendix C v1.3 — Platform Architecture (Operating Architecture, subdomain map, two data spines, Provider Selection Rationale)
- Appendix E v1.2 — The Atlas (28 assets with codes, phases, layers, purposes)
- AI Governance Framework v1.2 (Appendix A) — prohibitions, emotional safety, data governance, review cadence
- Brand Standards v1.2 (Appendix B; v1.3 in progress) — voice, vocabulary, capitalization, forbidden phrases
- Tom Miller Member Journey v4.2 (Appendix D, abridged; full master version internal) — canonical member experience
- Platform Development Plan v4 — phase sequencing, Consistency and Reliability Commitments, single platform launch (October 2026)
- G4L × Greg Welk Authoring Brief v1.0 — Greg's three sub-corpora authoring scope and cadence
- Fundraising Master Plan v1.8 — $450K raise, UFC fiscal sponsorship, entity architecture (Adjacent Lab LLC + GRINTA! Film + G4L Impact Campaign + UFC)
- Business Plan v6.1 (Update Pack v2 applied) — full program narrative

10.3 Document control

Version: 1.0

Date: May 2026

Status: Active. Multi-purpose working specification for developer hire, agency RFP, and internal team alignment. Built for iteration as Tom Miller Member Journey v4.2 evolves.

Owner: Jay Crain

Primary audience: Developer being hired, agency bidding on the build, internal team. Same document serves all three readers.

Next revision triggers:

- Tom Miller Member Journey v4.3 — point release for content refinement; Spec bumps to v1.1
- Tom Miller Member Journey v5.x — major change; Spec bumps to v2.x with architectural revisit
- Brand Standards v1.3 finalization — Layer 1 system prompt and Layer 2 voice slice update
- AI Governance Framework v1.3 — prohibition list, emotional safety protocol, retention policy update
- Appendix C v1.4 — reference architecture or two-data-spine update
- Material developer or agency feedback — open decisions resolved, defaults adjusted

Versioning convention recap:

- Tom v4.x → Spec v1.x (point release; section-level revision)
- Tom v5.x → Spec v2.x (major release; architectural revisit)

Companion deliverables:

- Member Agent Data Model SVG (front page)
- G4L × Greg Welk Authoring Brief v1.0 (parallel workstream)
- Future: Operating Moments diagram (optional second diagram showing what flows through each of the four operating moments)
- Future: Founder Agent Technical Spec (parallel architecture document for the founder-facing AI)